

scBatchProx: Federated-Inspired Refinement for Stable Cell-Type Discriminability under Heterogeneous Batch Compositions

Quang-Huy Nguyen, Jiaqi Wang*, Wei-Shinn Ku*

Department of Computer Science and Software Engineering

Auburn University

Auburn, AL 36849 USA

{hqn0001, jqwang, wz0004}@auburn.edu

Abstract

Single-cell integration workflows often construct low-dimensional cell embeddings and then refine them with post-hoc methods to reduce batch effects. This refinement process can become unstable when cell-type compositions vary across batches, with some populations underrepresented or absent in particular batches. The problem becomes more consequential in dynamic single-cell data systems, where newly acquired batches can change both technical conditions and cell-type composition. Such instability can reduce downstream cell-type classification performance and weaken stability under imbalance perturbations. We introduce `scBatchProx`, a lightweight post-hoc refinement method for stabilizing single-cell latent embeddings in these heterogeneous and evolving settings. `scBatchProx` operates directly on precomputed embeddings and treats each batch or study as a client in a federated-inspired optimization procedure. A batch-conditioned FiLM adapter learns local latent updates, while proximal and identity-preserving regularization keep these updates conservative. Experiments on multi-batch and cross-study single-cell datasets show that `scBatchProx` improves downstream cell-type classification across different upstream embeddings. In controlled imbalance perturbations, `scBatchProx` maintains more stable affected-cell-type F1 when selected populations are downsampled or ablated from one batch. In cumulative retraining and continual integration settings, `scBatchProx` remains effective as new datasets arrive over time. Together, these results suggest that conservative, federated-inspired refinement can help maintain stable single-cell embeddings as batch compositions change across datasets and over time.

1 Introduction

Single-cell RNA sequencing now profiles thousands to millions of cells in a single experiment (Klein et al. 2015; Macosko et al. 2015; Satpathy et al. 2019; Datlinger et al. 2021). As these datasets accumulate across studies, platforms, and laboratories, single-cell analysis increasingly depends on integrating cells into shared low-dimensional representations. This integration is difficult because technical variation can separate cells by batch rather than by biological state. If left uncorrected, these batch effects can dominate learned embeddings and distort downstream analyses.

Heterogeneous cell-type composition complicates the interpretation of integrated embeddings. Newly generated datasets

rarely contain the same populations in the same proportions. Some cell types may be enriched in one batch, depleted in another, or absent altogether. We refer to this variation as heterogeneous cell-type support. Uneven cell-type support can distort the representation of underrepresented populations and reduce their separability in the integrated embedding. These representation errors can consequently affect downstream cell-type classification (Maan et al. 2024).

Post-hoc refinement can improve existing embeddings without retraining the upstream embedding construction method. However, strong batch mixing and downstream discriminability are not the same objective. A refinement procedure can improve batch alignment while introducing latent changes that are unnecessary for preserving cell-type discriminability. The problem becomes more difficult as datasets evolve. Newly arriving batches can change both technical variation and cell-type composition, so repeated global correction may move previously stable embeddings. These settings call for a refinement strategy that adapts locally to each batch while limiting unnecessary movement in the latent space.

Federated learning (FL) provides a useful perspective for this problem. FL systems are designed to optimize across heterogeneous clients, each with its own local data distribution, while maintaining a shared global model. This structure closely mirrors single-cell integration under heterogeneous batch compositions, where each batch exhibits its own cell-type distribution and technical characteristics. It also naturally supports dataset evolution, since newly arriving clients can participate in optimization without revisiting historical data. We build on this correspondence in `scBatchProx`.

Model Overview. `scBatchProx` is a lightweight FL-inspired post-hoc framework for refining single-cell latent embeddings while preserving cell-type discriminability under heterogeneous and evolving batch compositions. As shown in Figure 1, `scBatchProx` takes as input a global cell-level embedding matrix produced by an arbitrary embedding construction method, with cells partitioned by batch from the same tissue. Each batch is treated as a local client, as in FL systems. A shared Feature-wise Linear Modulation (FiLM) (Perez et al. 2018) adapter applies batch-conditioned affine updates to the latent embedding. We choose FiLM for two reasons that are particularly relevant in evolving single-cell analysis pipelines. First, its scale-and-shift updates provide a conservative form of adaptation, helping preserve the

*Corresponding author.

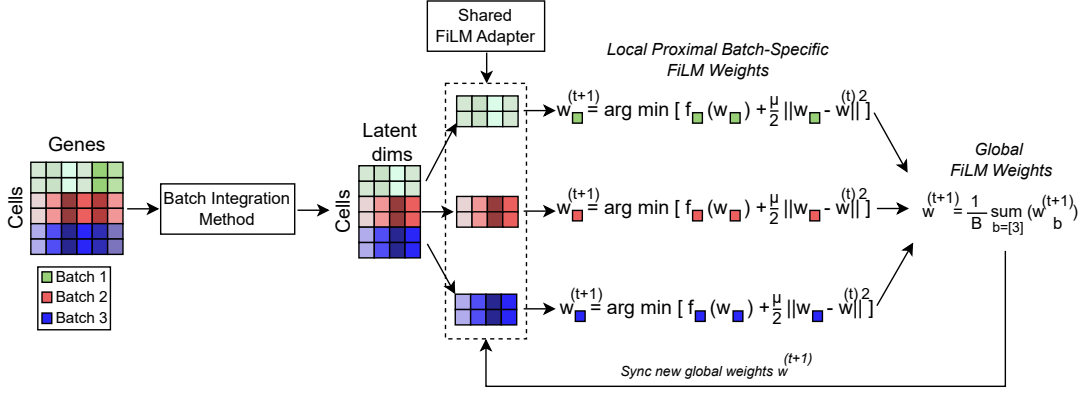


Figure 1: Framework of scBatchProx.

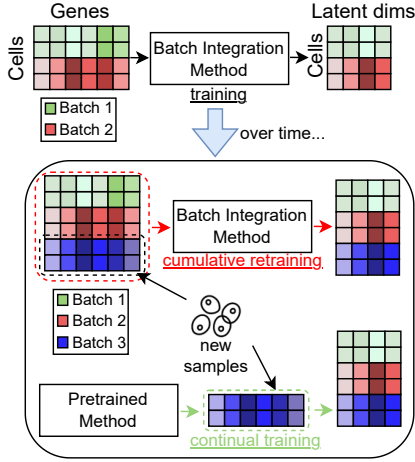


Figure 2: Two dataset evolution scenarios in single-cell analysis. Cumulative retraining recomputes embeddings by retraining the upstream model on all available data, whereas continual training fits newly arriving datasets into a fixed reference embedding without retraining on previously processed data.

existing embedding geometry as new batches are integrated over time. Second, FiLM introduces only a small number of trainable parameters, making repeated refinement lightweight, scalable, and deployable. Each client updates its local adapter using only its own embeddings, while a proximal penalty keeps the local update close to the current global adapter. An identity-preserving penalty further discourages unnecessary movement away from the upstream embedding. The updated adapters are aggregated across clients and broadcast for the next round.

Contributions. We make the following contributions:

- **Stable cell-type discriminability under heterogeneous cell-type support.** We study post-hoc single-cell embedding refinement in settings where batches contain uneven or incomplete cell-type support. scBatchProx refines precomputed embeddings through batch-conditioned FiLM adapters and conservative regularization. This de-

sign aims to preserve cell-type discriminability when specific populations are underrepresented or absent in some batches.

- **FL-inspired adaptation for evolving single-cell datasets.** The FL-inspired design of scBatchProx naturally accommodates heterogeneous cell-type support and dataset evolution. We evaluate scBatchProx in both *cumulative retraining* settings, where embeddings are periodically recomputed on all available data, and *continual training* settings, where new datasets are added to a fixed reference embedding. Figure 2 illustrates these two dataset evolution scenarios.
- **Lightweight and deployable post-hoc layer.** The FiLM parameterization keeps scBatchProx computationally lightweight and deployable on limited computational resources. This makes the refinement step practical for dynamic single-cell analysis pipelines in which new batches arrive over time.

2 Related Work

2.1 Cell-level embedding construction for single-cell integration

Single-cell integration workflows often begin by transforming high-dimensional gene expression profiles into low-dimensional cell embeddings. Many methods can produce such embeddings. Two representative strategies are batch-agnostic linear dimensionality reduction, exemplified by principal component analysis (PCA), and probabilistic batch-aware modeling, exemplified by single-cell variational inference (scVI) (Lopez et al. 2018). PCA provides a simple and efficient representation but does not model batch labels directly. In contrast, scVI learns a latent representation from count data while modeling batch effects within a probabilistic generative framework. These strategies define the upstream embedding space on which downstream correction or refinement is performed. However, end-to-end representation models such as scVI couple embedding construction with batch-aware modeling, so incorporating newly arriving datasets often requires retraining or reference-based adaptation. In this work, our focus is not embedding construction

itself, but post-hoc refinement of embeddings produced by arbitrary upstream methods.

2.2 Post-hoc integration and latent embedding correction

Harmony (Korsunsky et al. 2019) and scArches (Lotfollahi et al. 2022) are closely related to scBatchProx but address different parts of the integration workflow. Harmony performs post-hoc correction directly on precomputed embeddings. scArches focuses on mapping new query data into pretrained reference models through lightweight adaptation.

Post-hoc correction of precomputed embeddings. Harmony is a widely used method for embedding-level batch correction. It improves precomputed embeddings through iterative clustering and dataset-specific correction in latent space, without retraining the upstream representation model. Our setting differs in several important respects. Harmony is typically applied as a global correction procedure over the full embedding matrix. When new batches arrive, the usual workflow reruns correction on the updated collection of cells. This is less aligned with evolving datasets, where previously embedded cells should remain stable as new datasets arrive. In addition, our main focus is not only batch mixing, but also preserving cell-type discriminability when cell-type support varies across batches.

Reference-based adaptation and query mapping. scArches extends pretrained single-cell reference atlases through lightweight adaptation of pretrained reference models. While effective for query-to-reference mapping, it relies on compatible pretrained architectures, most commonly scVI and related variational autoencoder models. New data are incorporated through model-specific adaptation rather than direct refinement of existing embeddings. In contrast, scBatchProx operates directly on arbitrary precomputed embeddings and requires neither a pretrained reference model nor architecture-specific adaptation. This makes the refinement procedure applicable across different embedding backbones and naturally suited to settings where datasets evolve over time.

Overall, Harmony and scArches share two related limitations. First, neither is designed for settings where cell-type support varies across batches and new datasets arrive over time. Second, both remain tied to single-cell-specific correction or adaptation frameworks. scBatchProx addresses the first gap through an FL-inspired design that treats batches as heterogeneous local clients. It addresses the second by operating directly on embeddings and batch labels, allowing the same refinement principle to extend beyond single-cell data to heterogeneous embedding distributions.

3 Methodology

Given a precomputed cell-level embedding produced by an arbitrary upstream method, scBatchProx performs lightweight post-hoc refinement directly in latent space. The key idea is to perform conservative batch-specific refinement in latent space. To this end, we construct a moment-matched target for each batch and learn a batch-conditioned FiLM

adapter whose output approaches this target while remaining close to the identity map. The upstream embedding is fixed throughout training, and optimization is restricted to the adapter parameters.

3.1 Problem Setup

Let $Z \in \mathbb{R}^{N \times d}$ denote a fixed latent embedding matrix for N cells and latent dimension d . Let $b_i \in \{1, \dots, B\}$ be the batch label of cell i , and define $I_b = \{i : b_i = b\}$, $n_b = |I_b|$, and $Z_b = \{z_i : i \in I_b\}$. scBatchProx does not use the raw gene expression matrix or cell-type labels during adaptation. All optimization is performed using the fixed embedding Z , the batch labels, and moment targets computed in the same latent space.

For coordinate-wise operations, define the empirical global mean and standard deviation as

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^N z_i, \quad \hat{s} = \left(\frac{1}{N} \sum_{i=1}^N (z_i - \hat{\mu})^{\odot 2} \right)^{1/2},$$

and the corresponding batch-specific quantities as

$$\hat{\mu}_b = \frac{1}{n_b} \sum_{i \in I_b} z_i, \quad \hat{s}_b = \left(\frac{1}{n_b} \sum_{i \in I_b} (z_i - \hat{\mu}_b)^{\odot 2} \right)^{1/2}.$$

Here, powers, square roots, multiplication, and division are applied coordinate-wise. We use a small constant $\epsilon > 0$ for numerical stability and write $\hat{s}^\epsilon = \hat{s} + \epsilon$ and $\hat{s}_b^\epsilon = \hat{s}_b + \epsilon$.

Definition 3.1 (Latent moment refinement target). For each batch b , define a coordinate-wise scaling vector

$$r_b = \begin{cases} \hat{s}^\epsilon \oslash \hat{s}_b^\epsilon, & \text{if variance matching is enabled,} \\ 1, & \text{if only mean matching is used,} \end{cases}$$

where $\oslash$ denotes element-wise division. For a cell $i \in I_b$, the moment-matched target is

$$t_i = \hat{\mu} + r_b \odot (z_i - \hat{\mu}_b).$$

Equivalently, under mean-only matching, $t_i = z_i + (\hat{\mu} - \hat{\mu}_b)$. In our default implementation, variance matching is enabled and $\epsilon = 10^{-6}$.

The target t_i specifies a batch-level alignment objective in latent space. The learned adapter is encouraged to approach this target while remaining constrained by the adapter regularization.

This construction is compatible with a distributed implementation based on latent summary statistics and each client’s local embeddings. Each client only needs to provide n_b , $\sum_{i \in I_b} z_i$, and $\sum_{i \in I_b} z_i^{\odot 2}$ for the server to compute global latent moments. The server can then broadcast these moments back to clients for local target construction. Thus, target construction requires neither raw gene expression nor pairwise cell matching across batches.

Proposition 3.2 (Latent moment normalization properties).

For every batch b , the target embeddings $T_b = \{t_i : i \in I_b\}$ have empirical mean $\frac{1}{n_b} \sum_{i \in I_b} t_i = \hat{\mu}$. Moreover, their coordinate-wise empirical standard deviation is $\text{Std}(T_b) = r_b \odot \hat{s}_b$. Consequently, under variance matching, $\text{Std}(T_b) = \hat{s}_b \odot (\hat{s}^\epsilon \oslash \hat{s}_b^\epsilon)$, and in the idealized case $\epsilon = 0$ with nonzero batch standard deviations, $\text{Std}(T_b) = \hat{s}$ for all batches.

Proof. For any batch b ,

$$\frac{1}{n_b} \sum_{i \in I_b} t_i = \hat{\mu} + r_b \odot \left(\frac{1}{n_b} \sum_{i \in I_b} (z_i - \hat{\mu}_b) \right) = \hat{\mu}.$$

Since $t_i - \hat{\mu} = r_b \odot (z_i - \hat{\mu}_b)$, the coordinate-wise standard deviation of T_b is $|r_b| \odot \hat{s}_b$. The entries of r_b are nonnegative by construction, so this equals $r_b \odot \hat{s}_b$. The remaining statements follow by substituting the definition of r_b .

Proposition 3.3 (Within-batch affine preservation). *For any two cells $i, k \in I_b$, the target transformation satisfies $t_i - t_k = r_b \odot (z_i - z_k)$. Thus, the target construction aligns batch-level moments while preserving within-batch differences up to a coordinate-wise scaling.*

Proof. Subtracting the two target definitions gives $t_i - t_k = r_b \odot (z_i - \hat{\mu}_b) - r_b \odot (z_k - \hat{\mu}_b) = r_b \odot (z_i - z_k)$.

3.2 FiLM Adapter and Expressivity

`scBatchProx` uses a batch-conditioned FiLM adapter. Let $\Gamma \in \mathbb{R}^{B \times d}$ collect scale vectors γ_b , and let $\Delta \in \mathbb{R}^{B \times d}$ collect shift vectors β_b . The adapter parameters are $\Theta = (\Gamma, \Delta)$, and the refined embedding for cell i is $A_\Theta(z_i, b_i) = \gamma_{b_i} \odot z_i + \beta_{b_i}$. The implementation initializes $\gamma_b = \mathbf{1}$ and $\beta_b = \mathbf{0}$, so the initial adapter is exactly the identity transformation.

Lemma 3.4 (FiLM representability of the moment target). *The moment target belongs to the FiLM adapter class. In particular, for every batch b , define $\gamma_b^* = r_b$ and $\beta_b^* = \hat{\mu} - r_b \odot \hat{\mu}_b$. Then $A_{\Theta^*}(z_i, b_i) = t_i$ for all cells i .*

Proof. For any $i \in I_b$, $A_{\Theta^*}(z_i, b) = r_b \odot z_i + \hat{\mu} - r_b \odot \hat{\mu}_b = \hat{\mu} + r_b \odot (z_i - \hat{\mu}_b) = t_i$.

This lemma gives the basic mathematical reason why `scBatchProx` can perform latent refinement through lightweight batch-conditioned affine adaptation: the target moment correction is itself a diagonal affine map over latent coordinates, and the FiLM adapter parameterizes exactly this family of maps.

3.3 Latent Convex Refinement View

The above construction gives a useful way to interpret `scBatchProx`. The upstream representation model has already transformed the original high-dimensional expression profile into a fixed latent feature vector. Once these features are fixed, `scBatchProx` no longer optimizes a nonlinear representation model. It optimizes a small affine adapter on top of the frozen latent representation.

For each coordinate j and each cell $i \in I_b$, define $\theta_{bj} = \begin{bmatrix} \gamma_{bj} \\ \beta_{bj} \end{bmatrix}$ and $x_{ij} = \begin{bmatrix} z_{ij} \\ 1 \end{bmatrix}$.

Then the j -th coordinate of the adapter output can be written as $A_\Theta(z_i, b)_j = x_{ij}^\top \theta_{bj}$. Thus, after the embedding Z and targets T are fixed, fitting the FiLM adapter reduces to a regularized linear least-squares optimization problem over frozen latent features.

A centralized counterpart of the adapter objective can be written as the global latent refinement objective

$$\mathcal{F}(\Theta) = \sum_{b=1}^B \frac{n_b}{N} \left[\frac{\lambda_{\text{target}}}{n_b d} \sum_{i \in I_b} \|A_\Theta(z_i, b) - t_i\|_2^2 \right] + \frac{\lambda_{\text{id}}}{Bd} (\|\Gamma - \mathbf{1}\|_F^2 + \|\Delta\|_F^2).$$

The weights n_b/N make the data-fitting terms a sample-size-weighted average of client objectives, matching the aggregation weights used during federated optimization. The identity regularizer matches the implementation, where deviations from $\gamma_b = \mathbf{1}$ and $\beta_b = \mathbf{0}$ are penalized by the mean squared parameter deviation across all batch-indexed adapter entries.

Proposition 3.5 (Convex latent refinement objective). *The global objective $\mathcal{F}$ is a convex quadratic function of the adapter parameters $\Theta = (\Gamma, \Delta)$. If $\lambda_{\text{id}} > 0$, then $\mathcal{F}$ is strongly convex and has a unique minimizer.*

Proof. For fixed Z and T , each scalar prediction has the linear form $A_\Theta(z_i, b)_j = x_{ij}^\top \theta_{bj}$. Therefore, each target-fitting term is a squared affine function of the adapter parameters, hence a convex quadratic. The identity regularizer is also a convex quadratic and contributes positive curvature to every scale and shift coordinate when $\lambda_{\text{id}} > 0$. Therefore, the full objective is convex, and it is strongly convex under $\lambda_{\text{id}} > 0$.

This convexification applies only to the adapter stage. The upstream representation model remains unchanged throughout refinement. `scBatchProx` uses the upstream method to obtain fixed latent features and then solves a well-posed convex correction problem in that latent space. This distinction is important in heterogeneous federated settings: heterogeneous local updates are regularized in a low-dimensional adapter space rather than in the full nonconvex representation model.

3.4 Local Objective and Conservative Regularization

At communication round t , each client receives the current global adapter $\Theta^{(t)} = (\Gamma^{(t)}, \Delta^{(t)})$ and optimizes a local copy using its own data. For client b , the full-batch empirical local objective is

$$\mathcal{L}_b(\Theta; \Theta^{(t)}) = \frac{\lambda_{\text{target}}}{n_b d} \sum_{i \in I_b} \|A_\Theta(z_i, b) - t_i\|_2^2 + \mu \left\| \Theta - \Theta^{(t)} \right\|_F^2 + \frac{\lambda_{\text{id}}}{Bd} (\|\Gamma - \mathbf{1}\|_F^2 + \|\Delta\|_F^2),$$

where

$$\left\| \Theta - \Theta^{(t)} \right\|_F^2 = \left\| \Gamma - \Gamma^{(t)} \right\|_F^2 + \left\| \Delta - \Delta^{(t)} \right\|_F^2.$$

The first term fits the adapter output to the moment-matched target. The FedProx (Li et al. 2020) term keeps the local adapter close to the current global adapter. The identity term penalizes deviations from $\gamma_b = \mathbf{1}$ and $\beta_b = \mathbf{0}$. In implementation, each client carries a full local copy of the

adapter state. Only the data-fitting term is batch-specific. The proximal and identity terms regularize the full local copy before aggregation. Importantly, the moment correction is not enforced exactly in practice. The proximal and identity-preserving terms bias the adapter toward conservative latent changes, discouraging large shifts under heterogeneous batch compositions.

Proposition 3.6 (Well-posed local latent optimization). *If $\mu + \lambda_{\text{id}}/(Bd) > 0$, then $\mathcal{L}_b(\Theta; \Theta^{(t)})$ is a strongly convex quadratic function of the adapter parameters Θ . Therefore, each local subproblem has a unique minimizer.*

Proof. The map $A_\Theta(z_i, b)$ is affine in Θ . Hence, the target-fitting term is a convex quadratic function of Θ . The proximal and identity terms are positive quadratic penalties around fixed parameter values. Their combined Hessian contributes a positive multiple of the identity whenever $\mu + \lambda_{\text{id}}/(Bd) > 0$. Therefore, the full objective is strongly convex and has a unique minimizer.

The optimizer used in practice is Adam applied to mini-batches from each client loader. On each mini-batch, the target-fitting term is computed as a mean squared error over cells and latent dimensions, while the FedProx and identity terms are applied to the full local adapter state. Thus, the implementation performs stochastic iterative optimization of the above well-defined latent quadratic objective rather than updating the upstream embedding model.

3.5 Federated Optimization Procedure

Training proceeds in communication rounds. At the beginning of each round, the current global adapter $\Theta^{(t)}$ is copied to every client. Each client performs local optimization on triples (z_i, b_i, t_i) from its local data loader and returns the full updated adapter state, denoted $\Theta_b^{(t+1)}$. The server then applies sample-size-weighted federated averaging:

$$\Theta^{(t+1)} = \frac{1}{\sum_{b=1}^B n_b} \sum_{b=1}^B n_b \Theta_b^{(t+1)}.$$

This procedure follows the implementation in which each client loader uses the full local batch dataset with seeded shuffling, and client updates are weighted by the number of local cells during aggregation.

Theorem 3.7 (Conservative latent refinement by `scBatchProx`). *For fixed embeddings Z and moment targets T , `scBatchProx` defines a latent-space refinement procedure over FiLM adapter parameters. The FiLM adapter can represent the moment-matched target exactly. Under this target, each batch is aligned to the global latent mean. Under ideal variance matching with $\epsilon = 0$ and nonzero batch standard deviations, each batch also matches the global coordinate-wise standard deviation. The resulting fixed-feature objective is a convex quadratic in the adapter parameters. The proximal and identity-preserving penalties define strongly convex local objectives that bias client updates toward conservative latent changes before sample-size-weighted aggregation.*

Proof. The existence of an exact moment-target adapter follows from Lemma 3.4, or directly from the construction $\gamma_b^* = r_b$ and $\beta_b^* = \hat{\mu} - r_b \odot \hat{\mu}_b$. The aligned mean and standard deviation properties follow from Proposition 3.2. The fixed-feature global objective is convex quadratic by Proposition 3.5. The local optimization objective is strongly convex by Proposition 3.6. Therefore, `scBatchProx` implements a conservative latent refinement procedure by optimizing a regularized moment-alignment objective in the adapter space.

3.6 Inference and Post-hoc Refinement

After training, the learned adapter is applied once to the full embedding matrix. Given Z and batch labels $\{b_i\}_{i=1}^N$, the final refined embedding is $\tilde{Z} = \{\gamma_{b_i} \odot z_i + \beta_{b_i}\}_{i=1}^N$. The resulting embedding can be used directly in standard single-cell workflows, including visualization, cell-type classification, and integration benchmarking. Since `scBatchProx` only operates on precomputed latent embeddings and optimizes only FiLM adapter parameters, it remains compatible with arbitrary upstream representation methods.

4 Experiments

We evaluate whether `scBatchProx` stabilizes precomputed single-cell embeddings under heterogeneous and evolving batch compositions. The experiments address three questions. First, does `scBatchProx` improve downstream cell-type classification on imbalanced multi-batch and cross-study datasets? Second, does it preserve affected-cell-type discriminability when a population is downsampled or removed from one batch? Third, does it remain effective when datasets arrive over time in cumulative retraining and continual integration settings?

All experiments apply `scBatchProx` as a lightweight post-hoc stage on top of upstream embeddings. On each batch, local training uses Adam (Kingma and Ba 2015) with learning rate 5×10^{-2} for three local epochs per federated round. Each local objective includes a target-matching loss toward the precomputed latent refinement targets and a FedProx proximal penalty with $\mu = 10^{-3}$. We set $\lambda_{\text{target}} = 0.5$, $\lambda_{\text{id}} = 10^{-3}$, enable variance matching by default, and use $\epsilon = 10^{-6}$ for numerical stability. We use a batch size of 256, and each client loader uses the full local batch dataset with deterministic shuffling.

4.1 Benchmark

Multi-batch and Cross-study Datasets with Heterogeneous Cell-Type Support. We evaluate `scBatchProx` on datasets that contain both batch effects and heterogeneous cell-type support. The first setting captures within-study technical variation across experimental batches, such as repeated measurements or processing runs within a study. We refer to this setting as *multi-batch heterogeneity*. The second setting captures between-study variation from independently designed studies that may differ in protocol, platform, or laboratory. We refer to this setting as *cross-study heterogeneity*. Both settings become more difficult when cell-type support varies across batches. For multi-batch heterogeneity, we use

real scRNA-seq datasets from (Maan et al. 2024): the imbalanced four-batch PBMC dataset (Ding et al. 2020) and the six-batch mouse hindbrain development (MHD) dataset (Vladoiu et al. 2019). To evaluate cross-study heterogeneity, we use a two-batch Human Pancreas Multi-Study (HPMS) dataset.

For all datasets, we select 2,500 highly variable genes and normalize the data using Scanpy (Wolf, Angerer, and Theis 2018) by scaling each cell’s read counts to 10,000 followed by a $\log_{1p}$ transformation. For HPMS, we adopt the preprocessed version (Lotfollahi, Rybakov, and Naghipourfar 2025) used by the scVI–scArches pipeline, where gene filtering and highly variable gene selection have already been performed. Accordingly, we apply only normalization and $\log_{1p}$ transformation. Dataset summary statistics are reported in Table 4 of Appendix A.

Metrics. We use macro-F1 to evaluate downstream cell-type classification under varying cell-type support. We assess batch integration performance using the scIB benchmarking framework (Luecken et al. 2022), including biological conservation metrics (NMI, ARI, ASW, isolated-label F1, cLISI) and batch-removal metrics (batch ASW, iLISI, kBET, graph connectivity, and PCR). Full metric definitions and aggregation procedures are provided in Appendix B.

Baselines. We compare scBatchProx with Harmony and scArches for cell-type classification. We also evaluate whether scBatchProx can refine PCA and scVI embeddings under cumulative retraining and continual integration settings. For PCA, we fix the latent dimensionality to 40. For scVI, Harmony, and scArches, we use the default settings from their respective implementations.

4.2 Results

Cell-Type Classification under Heterogeneous Batch Composition. We train a balanced logistic regression classifier on the refined embeddings to predict cell-type labels. A fixed stratified train/test split (80% / 20%) is used across all models to ensure fair comparison.

Table 1 shows that scBatchProx improves macro-F1 across all datasets and upstream embeddings. The largest gains occur on MHD, where cell-type support differs most strongly across batches. This suggests that conservative refinement becomes increasingly useful as composition heterogeneity increases. Notably, the improvement persists on both PCA and scVI embeddings, indicating that the refinement effect is not tied to a specific upstream representation.

Stability under Imbalanced Batch Compositions. To evaluate stability under reduced cell-type support, we use a balanced two-batch PBMC dataset derived from (Ding et al. 2020) and used in (Maan et al. 2024), containing six shared cell populations with 200 cells per cell type in each batch. We perturb one selected cell type in one batch using two regimes: **downsample**, where only 10% of cells are retained, and **ablation**, where the population is removed entirely. We report affected-cell-type F1, defined as the F1 score computed only on the perturbed cell type after post-hoc refinement of PCA embeddings.

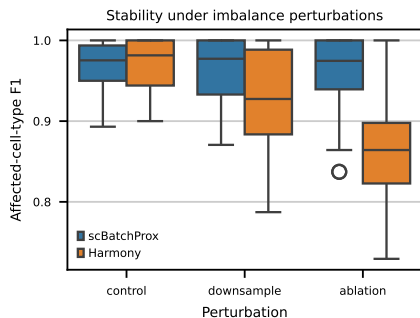


Figure 3: Stability under imbalance perturbations measured using affected-cell-type F1 on post-hoc refinement of PCA embeddings. scBatchProx maintains substantially more stable affected-cell-type F1 across perturbation regimes.

As shown in Figure 3, under balanced conditions, both Harmony and scBatchProx achieve similarly strong affected-cell-type F1 scores. However, as cell-type support becomes reduced or entirely absent within one batch, Harmony exhibits substantially larger degradation, particularly in the ablation setting. In contrast, scBatchProx maintains substantially more stable performance across perturbation regimes, suggesting that conservative latent refinement better preserves affected-cell-type discriminability in this perturbation setting.

Dataset Evolution. We next ask whether scBatchProx remains effective as datasets evolve over time. We first evaluate continual training, where newly arriving datasets are incorporated without retraining on previously processed data. The initial protocols define a reference embedding, and later datasets are added incrementally. Figure 4 shows that scBatchProx consistently improves aggregate scIB scores across both protocol arrival orders, indicating that conservative latent refinement remains effective under continual dataset growth.

We additionally evaluate cumulative retraining, where all available datasets are re-embedded whenever new data arrive. Similar improvements are observed across both protocol arrival orders (Figure 5 of Appendix C). Runtime comparisons in Table 5 of Appendix C further show that cumulative runtime grows substantially faster for workflows that repeatedly recompute upstream embeddings, whereas scBatchProx adds only a lightweight post-hoc refinement stage.

Beyond Single-Cell Embeddings. We demonstrate the broader applicability of scBatchProx beyond single-cell data using the QM8 dataset (Wu et al. 2026) from MoleculeNet (Wu et al. 2018). This dataset consists of small molecules with quantum mechanical targets. Molecules are converted from SMILES strings into fixed-length feature vectors using Morgan fingerprints (Morgan 1965). To simulate heterogeneous and imbalanced latent distributions, we partition samples based on heavy atom count, resulting in structurally distinct groups with substantially different sample support (see Appendix D, Table 6). These fingerprint representations are treated as embeddings for post-hoc refinement.

We evaluate discriminability across these structurally

Upstream Method	Model	4-batch PBMC		6-batch MHD		2-batch HPMS	
		Macro-F1	Runtime (s)	Macro-F1	Runtime (s)	Macro-F1	Runtime (s)
PCA	Harmony	0.6633 $\pm$ 0.0044	14.2332 $\pm$ 1.7650	0.3829 $\pm$ 0.0023	3.2045 $\pm$ 0.3022	0.8706 $\pm$ 0.0007	21.2656 $\pm$ 1.7654
	scBatchProx	0.7594 $\pm$ 0.0008	5.3492 $\pm$ 0.9618	0.6567 $\pm$ 0.001	1.98 $\pm$ 0.2187	0.8710 $\pm$ 0.0010	5.4333 $\pm$ 0.5713
scVI	scArches	0.4415 $\pm$ 0.0000	16.6963 $\pm$ 2.7281	0.1643 $\pm$ 0.0000	14.8221 $\pm$ 0.2786	0.7745 $\pm$ 0.0000	38.0826 $\pm$ 0.5940
	Harmony	0.7010 $\pm$ 0.0071	7.1867 $\pm$ 0.9681	0.2683 $\pm$ 0.0051	5.9330 $\pm$ 4.3050	0.8502 $\pm$ 0.002	12.0450 $\pm$ 1.7893
	scBatchProx	0.7504 $\pm$ 0.0001	0.5540 $\pm$ 0.1382	0.4747 $\pm$ 0.0002	18.4768 $\pm$ 0.6348	0.8613 $\pm$ 0.0002	0.8574 $\pm$ 0.1798

Table 1: Measurement of cell-type classification performance using macro-F1 score (the higher, the better). All results are reported as mean $\pm$ standard deviation over 5 independent runs. scBatchProx improves downstream classification performance across datasets and embedding backbones, with competitive runtime in most settings.

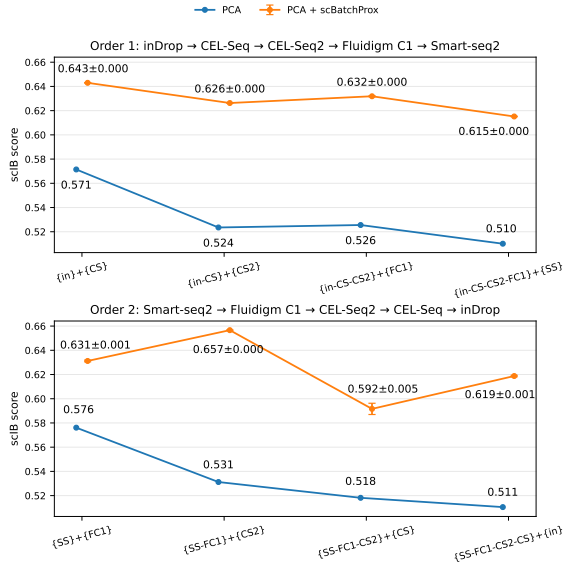


Figure 4: Aggregate scIB scores for continual training under protocol arrival: (a) technological progression order (inDrop → CEL-Seq → CEL-Seq2 → Fluidigm C1 → Smart-seq2) and (b) reverse order (Smart-seq2 → Fluidigm C1 → CEL-Seq2 → CEL-Seq → inDrop). Continual training is commonly encountered in practice, yet most end-to-end batch-aware models do not support it. Despite this constraint, scBatchProx consistently improves embedding quality across both arrival orders.

Upstream Method	Model	QM8	
		Macro-F1	Runtime (s)
Morgan	Harmony	0.3065 $\pm$ 0.0108	22.6302 $\pm$ 6.1664
	scBatchProx	0.6996 $\pm$ 0.0000	8.29 $\pm$ 2.26

Table 2: Representation quality on QM8 measured by macro-F1 on classification over structurally defined groups.

defined groups using macro-F1. As shown in Table 2, scBatchProx substantially outperforms Harmony on this task. Notably, the setting contains neither gene-expression measurements nor cell-type annotations, indicating that the benefits of conservative refinement are not tied to single-cell-specific assumptions. This result supports the broader

Table 3: Ablation study measured using macro-F1 score (higher is better). All results are reported as mean $\pm$ standard deviation over 5 independent runs.

Variant	4-batch PBMC	6-batch MHD	2-batch HPMS
Full model	0.7585 $\pm$ 0.0009	0.6140 $\pm$ 0.0006	0.8725 $\pm$ 0.0005
mean-only	0.7478 $\pm$ 0.0006	0.6173 $\pm$ 0.0004	0.8718 $\pm$ 0.0006
w/o FedProx	0.7471 $\pm$ 0.0000	0.6571 $\pm$ 0.0007	0.8741 $\pm$ 0.0011
w/o identity	0.7577 $\pm$ 0.0000	0.6121 $\pm$ 0.0011	0.8716 $\pm$ 0.0013

applicability of scBatchProx to heterogeneous embedding distributions beyond single-cell data.

5 Ablation study

The ablation results show that the contribution of each regularization component depends on dataset heterogeneity. Identity-preserving regularization is consistently beneficial, whereas the effect of FedProx varies across datasets. In particular, removing FedProx improves performance on MHD and HPMS, suggesting that highly heterogeneous datasets may benefit from more flexible local adaptation. Mean-only refinement generally underperforms the full model, supporting the use of variance-aware moment refinement.

6 Conclusion

We introduced scBatchProx, a lightweight post-hoc latent refinement framework for stabilizing single-cell embeddings under heterogeneous and evolving batch compositions. By operating directly on precomputed embeddings, scBatchProx improves downstream cell-type classification and maintains more stable affected-cell-type discriminability under reduced cell-type support. The method remains effective in both cumulative retraining and continual integration settings while requiring only lightweight adapter optimization. Future work will investigate when post-hoc refinement becomes preferable to repeated global retraining and whether atlas-scale integration can be decomposed into many smaller refinement tasks under limited computational resources.

Acknowledgments

We acknowledge the Auburn University Easley Cluster for support of this work.

References

- Datlinger, P.; Rendeiro, A. F.; Boenke, T.; Senekowitsch, M.; Krausgruber, T.; Barreca, D.; and Bock, C. 2021. Ultra-high-throughput single-cell RNA sequencing and perturbation screening with combinatorial fluidic indexing. *Nature Methods*, 18(6): 635–642.
- Ding, J.; Adiconis, X.; Simmons, S. K.; Kowalczyk, M. S.; Hession, C. C.; Marjanovic, N. D.; Hughes, T. K.; Wadsworth, M. H.; Burks, T.; Nguyen, L. T.; Kwon, J. Y. H.; Barak, B.; Ge, W.; Kedaigle, A. J.; Carroll, S.; Li, S.; Hacohen, N.; Rozenblatt-Rosen, O.; Shalek, A. K.; Villani, A.-C.; Regev, A.; and Levin, J. Z. 2020. Systematic comparison of single-cell and single-nucleus RNA-sequencing methods. *Nature Biotechnology*, 38(6): 737–746.
- Kingma, D. P.; and Ba, J. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015*. San Diego, CA, USA.
- Klein, A. M.; Mazutis, L.; Akartuna, I.; Tallapragada, N.; Veres, A.; Li, V.; Peshkin, L.; Weitz, D. A.; and Kirschner, M. W. 2015. Droplet barcoding for single-cell transcriptomics applied to embryonic stem cells. *Cell*, 161(5): 1187–1201.
- Korsunsky, I.; Millard, N.; Fan, J.; Slowikowski, K.; Zhang, F.; Wei, K.; Baglaenko, Y.; Brenner, M.; Loh, P.-r.; and Raychaudhuri, S. 2019. Fast, sensitive and accurate integration of single-cell data with Harmony. *Nature Methods*, 16: 1289–1296.
- Li, T.; Sahu, A. K.; Zaheer, M.; Sanjabi, M.; Talwalkar, A.; and Smith, V. 2020. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems*, volume 2, 429–450. Austin, TX, USA: mlsys.org.
- Lopez, R.; Regier, J.; Cole, M. B.; Jordan, M. I.; and Yosef, N. 2018. Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15: 1053–1058.
- Lotfollahi, M.; Naghipourfar, M.; Luecken, M. D.; Khajavi, M.; Büttner, M.; Wagenstetter, M.; Avsec, Ž.; Gayoso, A.; Yosef, N.; Interlandi, M.; Rybakov, S.; Misharin, A. V.; and Theis, F. J. 2022. Mapping single-cell data to reference atlases by transfer learning. *Nature Biotechnology*, 40: 121–130.
- Lotfollahi, M.; Rybakov, S.; and Naghipourfar, M. 2025. Un-supervised surgery pipeline with SCVI. https://docs.scarches.org/en/latest/scvi_surgery_pipeline.html. Accessed: 2026-05-29.
- Luecken, M. D.; Büttner, M.; Chaichoompu, K.; Danese, A.; Interlandi, M.; Müller, M. F.; Strobl, D. C.; Zappia, L.; Dugas, M.; Colomé-Tatché, M.; and Theis, F. J. 2022. Benchmarking atlas-level data integration in single-cell genomics. *Nature Methods*, 19: 41–50.
- Maan, H.; Zhang, L.; Yu, C.; Geuenich, M. J.; Campbell, K. R.; and Wang, B. 2024. Characterizing the impacts of dataset imbalance on single-cell data integration. *Nature Biotechnology*, 42: 1899–1908.
- Macosko, E. Z.; Basu, A.; Satija, R.; Nemesh, J.; Shekhar, K.; Goldman, M.; Tirosh, I.; Bialas, A. R.; Kamitaki, N.; Martersteck, E. M.; Trombetta, J. J.; Weitz, D. A.; Sanes, J. R.; Shalek, A. K.; Regev, A.; and McCarroll, S. A. 2015. Highly parallel genome-wide expression profiling of individual cells using nanoliter droplets. *Cell*, 161(5): 1202–1214.
- Morgan, H. L. 1965. The Generation of a Unique Machine Description for Chemical Structures-A Technique Developed at Chemical Abstracts Service. *Journal of Chemical Documentation*, 5(2): 107–113.
- Perez, E.; Strub, F.; De Vries, H.; Dumoulin, V.; and Courville, A. 2018. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence*, volume 32, 3942–3951. New Orleans, Louisiana, USA: AAAI Press.
- Satpathy, A. T.; Granja, J. M.; Yost, K. E.; Qi, Y.; Meschi, F.; McDermott, G. P.; Olsen, B. N.; Mumbach, M. R.; Pierce, S. E.; Corces, M. R.; Shah, P.; Bell, J. C.; Jhutti, D.; Nemec, C. M.; Wang, J.; Wang, L.; Yin, Y.; Giresi, P. G.; Chang, A. L. S.; Zheng, G. X. Y.; Greenleaf, W. J.; and Chang, H. Y. 2019. Massively parallel single-cell chromatin landscapes of human immune cell development and intratumoral T cell exhaustion. *Nature Biotechnology*, 37: 925–936.
- Vladoiu, M. C.; El-Hamamy, I.; Donovan, L. K.; Farooq, H.; Holgado, B. L.; Sundaravadanam, Y.; Ramaswamy, V.; Hendrikse, L. D.; Kumar, S.; Mack, S. C.; Lee, J. J. Y.; Fong, V.; Juraschka, K.; Przelicki, D.; Michealraj, A.; Skowron, P.; Luu, B.; Suzuki, H.; Morrissy, A. S.; Cavalli, F. M. G.; Garzia, L.; Daniels, C.; Wu, X.; Qazi, M. A.; Singh, S. K.; Chan, J. A.; Marra, M. A.; Malkin, D.; Dirks, P.; Heisler, L.; Pugh, T.; Ng, K.; Notta, F.; Thompson, E. M.; Kleinman, C. L.; Joyner, A. L.; Jabado, N.; Stein, L.; and Taylor, M. D. 2019. Childhood cerebellar tumours mirror conserved fetal transcriptional programs. *Nature*, 572(7767): 67–73.
- Wolf, F. A.; Angerer, P.; and Theis, F. J. 2018. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology*, 19: 15.
- Wu, Z.; Ramsundar, B.; Feinberg, E. N.; Gomes, J.; Geniesse, C.; Pappu, A. S.; and Leswing, K. 2026. Dataset Collection. <https://moleculenet.org/datasets-1>. Accessed: 2026-05-29.
- Wu, Z.; Ramsundar, B.; Feinberg, E. N.; Gomes, J.; Geniesse, C.; Pappu, A. S.; Leswing, K.; and Pande, V. 2018. MoleculeNet: A Benchmark for Molecular Machine Learning. arXiv:1703.00564.

A Data Statistics

Table 4 summarizes the statistics of all datasets used in this work, including the batch-wise cell-type compositions to illustrate the degree of compositional imbalance across batches. Among the three datasets, 6-batch MHD exhibits the most severe heterogeneity in cell-type distributions, with several cell types appearing exclusively or predominantly in specific batches.

B METRICS – DEFINITIONS AND INTERPRETATION

We evaluate data integration performance using the scIB benchmarking framework (Luecken et al. 2022), which provides a standardized collection of quantitative metrics to

Dataset	Batch	# Cells	# Cells per Cell Type
4-batch PBMC	10x	3,222	Cytotoxic T: 962; CD4+ T: 960; B: 346; NK: 194; CD14+ mono: 354; CD16+ mono: 98; DC: 38; pDC: 0; Megakaryocyte: 270; Unassigned: 0
	CEL	526	Cytotoxic T: 174; CD4+ T: 160; B: 80; NK: 42; CD14+ mono: 31; CD16+ mono: 21; DC: 0; pDC: 0; Megakaryocyte: 18; Unassigned: 0
	SeqWell	3,773	Cytotoxic T: 1,278; CD4+ T: 566; B: 527; NK: 0; CD14+ mono: 1,255; CD16+ mono: 0; DC: 37; pDC: 26; Megakaryocyte: 38; Unassigned: 46
	SmartSeq2	526	Cytotoxic T: 193; CD4+ T: 112; B: 79; NK: 50; CD14+ mono: 60; CD16+ mono: 18; DC: 0; pDC: 0; Megakaryocyte: 14; Unassigned: 0
6-batch MHD	e8.0_sce31053	5,496	Hindbrain glutamatergic: 0; Mesenchyme: 95; Dorsal hindbrain: 1,843; Gliogenic progenitors: 0; VZ progenitors: 0; Caudal: 1,218; Hindbrain roof plate: 1,204; Neural crest: 452; Hindbrain GABAergic: 0; Early ectoderm: 748; Proliferating VZ progenitors: 0; Purkinje cells: 0; Erythrocytes and erythrocyte progenitors: 0; Anterior: 695; Hindbrain: 0; Endothelial cells: 0; Sensory neuron: 0; Extraembryonic endoderm: 342; Endoderm: 187
	e9.0_sce31053	5,103	Hindbrain glutamatergic: 4; Mesenchyme: 1,624; Dorsal hindbrain: 0; Gliogenic progenitors: 0; VZ progenitors: 0; Caudal: 38; Hindbrain roof plate: 0; Neural crest: 240; Hindbrain GABAergic: 0; Early ectoderm: 51; Proliferating VZ progenitors: 0; Purkinje cells: 0; Erythrocytes and erythrocyte progenitors: 0; Anterior: 2; Hindbrain: 0; Endothelial cells: 0; Sensory neuron: 639; Extraembryonic endoderm: 0; Endoderm: 0
	e10.0_sce31053	6,606	Hindbrain glutamatergic: 62; Mesenchyme: 1,825; Dorsal hindbrain: 0; Gliogenic progenitors: 0; VZ progenitors: 0; Caudal: 0; Hindbrain roof plate: 0; Neural crest: 240; Hindbrain GABAergic: 0; Early ectoderm: 0; Proliferating VZ progenitors: 0; Purkinje cells: 0; Erythrocytes and erythrocyte progenitors: 159; Anterior: 0; Hindbrain: 0; Endothelial cells: 0; Sensory neuron: 0; Extraembryonic endoderm: 0; Endoderm: 0
	E10_sce27998	7,725	Hindbrain glutamatergic: 4,758; Mesenchyme: 0; Dorsal hindbrain: 0; Gliogenic progenitors: 0; VZ progenitors: 1,521; Caudal: 0; Hindbrain roof plate: 0; Neural crest: 0; Hindbrain GABAergic: 70; Early ectoderm: 0; Proliferating VZ progenitors: 0; Purkinje cells: 737; Erythrocytes and erythrocyte progenitors: 0; Anterior: 0; Hindbrain: 670; Endothelial cells: 667; Sensory neuron: 0; Extraembryonic endoderm: 0; Endoderm: 0
	E12_sce27998	5,460	Hindbrain glutamatergic: 3,710; Mesenchyme: 0; Dorsal hindbrain: 0; Gliogenic progenitors: 36; VZ progenitors: 2; Caudal: 0; Hindbrain roof plate: 0; Neural crest: 0; Hindbrain GABAergic: 715; Early ectoderm: 0; Proliferating VZ progenitors: 703; Purkinje cells: 32; Erythrocytes and erythrocyte progenitors: 0; Anterior: 0; Hindbrain: 0; Endothelial cells: 0; Sensory neuron: 0; Extraembryonic endoderm: 0; Endoderm: 0
	E14_sce27998	3,730	Hindbrain glutamatergic: 1,089; Mesenchyme: 0; Dorsal hindbrain: 0; Gliogenic progenitors: 1,739; VZ progenitors: 0; Caudal: 0; Hindbrain roof plate: 0; Neural crest: 0; Hindbrain GABAergic: 49; Early ectoderm: 0; Proliferating VZ progenitors: 70; Purkinje cells: 0; Erythrocytes and erythrocyte progenitors: 0; Anterior: 0; Hindbrain: 0; Endothelial cells: 0; Sensory neuron: 0; Extraembryonic endoderm: 0; Endoderm: 0
2-batch HPMS	0	12,720	Beta: 4,289; Alpha: 3,595; Ductal: 1,642; Delta: 899; Endothelial: 769; Acinar: 610; Gamma: 418; Stellate: 498
	1	2,961	Beta: 796; Alpha: 1,109; Ductal: 462; Delta: 142; Endothelial: 67; Acinar: 103; Gamma: 219; Stellate: 63

Table 4: Cell-type composition imbalance across batches

assess the trade-off between batch-effect removal and biological signal preservation. All metrics are computed on the low-dimensional integrated representations produced by each method unless stated otherwise.

B.1 Biological Conservation Metrics

Biological conservation metrics quantify how well biologically meaningful structure, in particular cell-type identity and local neighborhood relationships, is preserved after integration.

Clustering-based label agreement (NMI, ARI). We assess global agreement between unsupervised clustering results and ground-truth cell-type annotations using normalized mutual information (NMI) and adjusted Rand index (ARI). Let C denote the clustering assignment obtained by applying K-means to the integrated embedding and Y the true cell-type labels. NMI is defined as

$$\text{NMI}(C, Y) = \frac{2I(C; Y)}{H(C) + H(Y)},$$

where $I(\cdot; \cdot)$ denotes mutual information and $H(\cdot)$ denotes entropy. ARI measures pairwise agreement between C and Y , adjusted for chance, with values closer to 1 indicating better recovery of cell-type structure.

Cell-type average silhouette width (ASW). The silhouette width for a cell i is defined as

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}},$$

where $a(i)$ is the average distance from i to other cells of the same cell type and $b(i)$ is the minimum average distance to cells of a different cell type. The cell-type ASW is computed by averaging $s(i)$ across all cells and rescaled to $[0, 1]$ as

$$\text{ASW}_{\text{cell}} = \frac{\text{ASW} + 1}{2},$$

with higher values indicating better separation between cell types.

Isolated-label F1 score. The isolated-label F1 score evaluates preservation of rare or batch-specific cell types that appear in only a subset of batches. For each isolated label, precision and recall are computed based on neighborhood consistency, and the F1 score is defined as

$$\text{F1} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

Higher values indicate improved conservation of rare cell populations.

Cell-type local inverse Simpson's index (cLISI). cLISI measures the diversity of cell-type labels within local neighborhoods of the integrated embedding. For a cell i , cLISI is defined as

$$\text{cLISI}(i) = \left(\sum_c p_{i,c}^2 \right)^{-1},$$

where $p_{i,c}$ denotes the proportion of neighbors of i belonging to cell type c . Lower cLISI values indicate stronger local cell-type purity and better biological conservation.

B.2 Batch-Effect Removal Metrics

Batch-effect removal metrics quantify how effectively technical variation across batches or datasets is mitigated while avoiding overcorrection.

Batch average silhouette width (batch ASW). Batch ASW is computed analogously to cell-type ASW, but using batch labels instead of cell-type labels. For each cell i , the absolute silhouette width $|s(i)|$ is computed and rescaled such that

$$\text{ASW}_{\text{batch}} = 1 - |s(i)|.$$

Higher values indicate better batch mixing.

Integration local inverse Simpson's index (iLISI). iLISI evaluates batch diversity within local neighborhoods. For a cell i ,

$$\text{iLISI}(i) = \left(\sum_b p_{i,b}^2 \right)^{-1},$$

where $p_{i,b}$ denotes the proportion of neighbors from batch b . Higher iLISI values indicate stronger batch mixing.

kBET per label. The k-nearest neighbor batch effect test (kBET) evaluates whether batch labels are uniformly distributed within local neighborhoods. In the per-label variant, kBET is applied separately within each cell type. The final score is the average acceptance rate across labels, with higher values indicating reduced batch effects.

Graph connectivity. Graph connectivity measures whether cells of the same cell type form a connected component across batches in a k-nearest neighbor graph constructed on the integrated representation. Let G_c denote the subgraph induced by cells of cell type c . The connectivity score is defined as the fraction of nodes belonging to the largest connected component of G_c , averaged across cell types. Higher values indicate successful integration without fragmenting biological structure.

Principal component regression (PCR). PCR quantifies the fraction of variance in the integrated embedding explained by batch labels. Let Z denote the integrated representation and S the batch labels. PCR is computed by regressing principal components of Z on S , and the coefficient of determination R^2 is averaged across components. Lower PCR values indicate reduced batch-associated variance.

B.3 Aggregate Scores

To summarize integration performance, scIB aggregates individual metrics into composite scores. All metrics are first scaled to the unit interval $[0, 1]$, with higher values indicating better performance.

The biological conservation score is computed as the arithmetic mean of all biological conservation metrics,

$$\text{Bio} = \frac{1}{|\mathcal{M}_{\text{bio}}|} \sum_{m \in \mathcal{M}_{\text{bio}}} m,$$

where $\mathcal{M}_{\text{bio}}$ includes NMI, ARI, cell-type ASW, isolated-label F1, and cLISI.

Similarly, the batch-effect removal score is computed as

$$\text{Batch} = \frac{1}{|\mathcal{M}_{\text{batch}}|} \sum_{m \in \mathcal{M}_{\text{batch}}} m,$$

where $\mathcal{M}_{\text{batch}}$ includes batch ASW, iLISI, kBET per label, graph connectivity, and PCR.

Following scIB conventions, the overall integration score is computed as a weighted mean,

$$\text{Overall} = 0.6 \cdot \text{Bio} + 0.4 \cdot \text{Batch},$$

placing greater emphasis on biological conservation while still encouraging effective batch-effect removal.

C Cumulative Retraining Under Dataset Evolution.

Figure 5 illustrates the batch mixing performance of `scBatchProx` for cumulative retraining under protocol arrival. Table 5 shows wall-clock runtime comparison for this setting.

D Synthetic Heterogeneous Batch Construction in QM8.

We construct synthetic batches by partitioning molecules based on heavy atom count, thereby grouping molecules by structural complexity. This results in highly imbalanced and heterogeneous batches (Table 6), reflecting realistic distribution shifts in which certain molecular regimes are underrepresented, with the majority of molecules concentrated in the high-complexity regime. Consequently, batch membership is strongly correlated with molecular structure and feature distribution, creating a challenging setting for post hoc batch correction.

Table 6: Synthetic batch construction on QM8 based on heavy atom count, resulting in highly imbalanced and heterogeneous groups.

Batch	# Molecules	Heavy Atom Range
0	17	≤ 3
1	158	4–5
2	21,611	≥ 6

Ord.	Method	Step	Run	Total	Cum.
1	PCA	1	0.16	154.91	0.16
1	PCA	2	0.24	236.19	0.41
1	PCA	3	0.45	320.40	0.86
1	PCA	4	0.72	366.72	1.58
1	PCA— <code>scBatchProx</code>	1	3.35	14.93	3.35
1	PCA— <code>scBatchProx</code>	2	4.55	18.24	7.90
1	PCA— <code>scBatchProx</code>	3	3.20	18.44	11.10
1	PCA— <code>scBatchProx</code>	4	12.20	18.04	23.30
1	scVI	1	154.54	154.91	154.54
1	scVI	2	235.65	236.19	390.20
1	scVI	3	319.61	320.40	709.81
1	scVI	4	365.65	366.72	1075.45
1	scVI— <code>scBatchProx</code>	1	11.46	14.93	11.46
1	scVI— <code>scBatchProx</code>	2	13.58	18.24	25.04
1	scVI— <code>scBatchProx</code>	3	15.11	18.44	40.15
1	scVI— <code>scBatchProx</code>	4	5.71	18.04	45.86
2	PCA	1	0.10	94.20	0.10
2	PCA	2	0.16	155.35	0.26
2	PCA	3	0.26	185.90	0.52
2	PCA	4	0.78	397.56	1.31
2	PCA— <code>scBatchProx</code>	1	3.75	9.08	3.75
2	PCA— <code>scBatchProx</code>	2	5.80	12.00	9.55
2	PCA— <code>scBatchProx</code>	3	5.95	10.70	15.49
2	PCA— <code>scBatchProx</code>	4	11.14	19.56	26.63
2	scVI	1	93.98	94.20	93.98
2	scVI	2	155.00	155.35	248.98
2	scVI	3	185.44	185.90	434.42
2	scVI	4	396.40	397.56	830.82
2	scVI— <code>scBatchProx</code>	1	5.15	9.08	5.15
2	scVI— <code>scBatchProx</code>	2	6.08	12.00	11.22
2	scVI— <code>scBatchProx</code>	3	4.63	10.70	15.86
2	scVI— <code>scBatchProx</code>	4	8.25	19.56	24.10

Table 5: Wall-clock runtime comparison under cumulative dataset growth. We report per-step time (Run), total retraining cost (Total), and cumulative runtime (Cum.) across sequential data integration steps for baseline methods (PCA, scVI) and post hoc refinement (`scBatchProx`). Order 1 corresponds to the technological progression order, and Order 2 corresponds to the reverse order. The reported total time (i.e., full wall-clock time) includes data loading, preprocessing (e.g., data normalization, $\log_{1p}$ transformation, etc.), training, and inference.

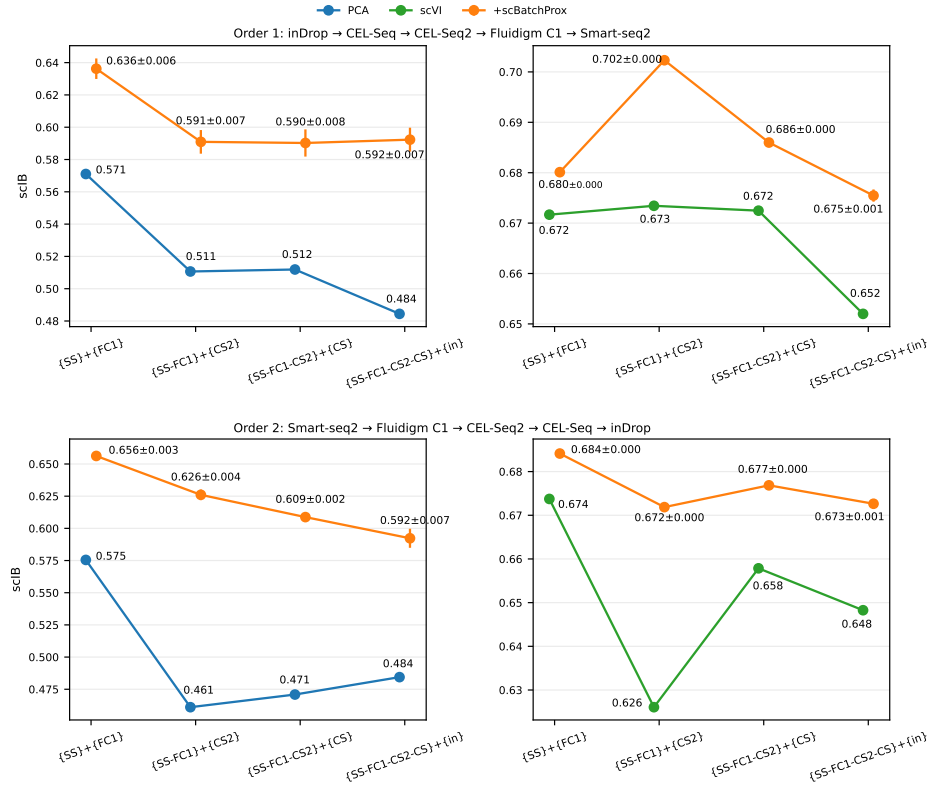


Figure 5: Aggregate scIB scores for cumulative retraining under protocol arrival: (a) technological progression order (inDrop → CEL-Seq → CEL-Seq2 → Fluidigm C1 → Smart-seq2) and (b) reverse order (Smart-seq2 → Fluidigm C1 → CEL-Seq2 → CEL-Seq → inDrop). While cumulative retraining is supported by most existing methods, it is computationally expensive. scBatchProx consistently improves overall embedding quality across both arrival orders.